



TeamFlow Systems

Unified Project Management & Incident Response Platform

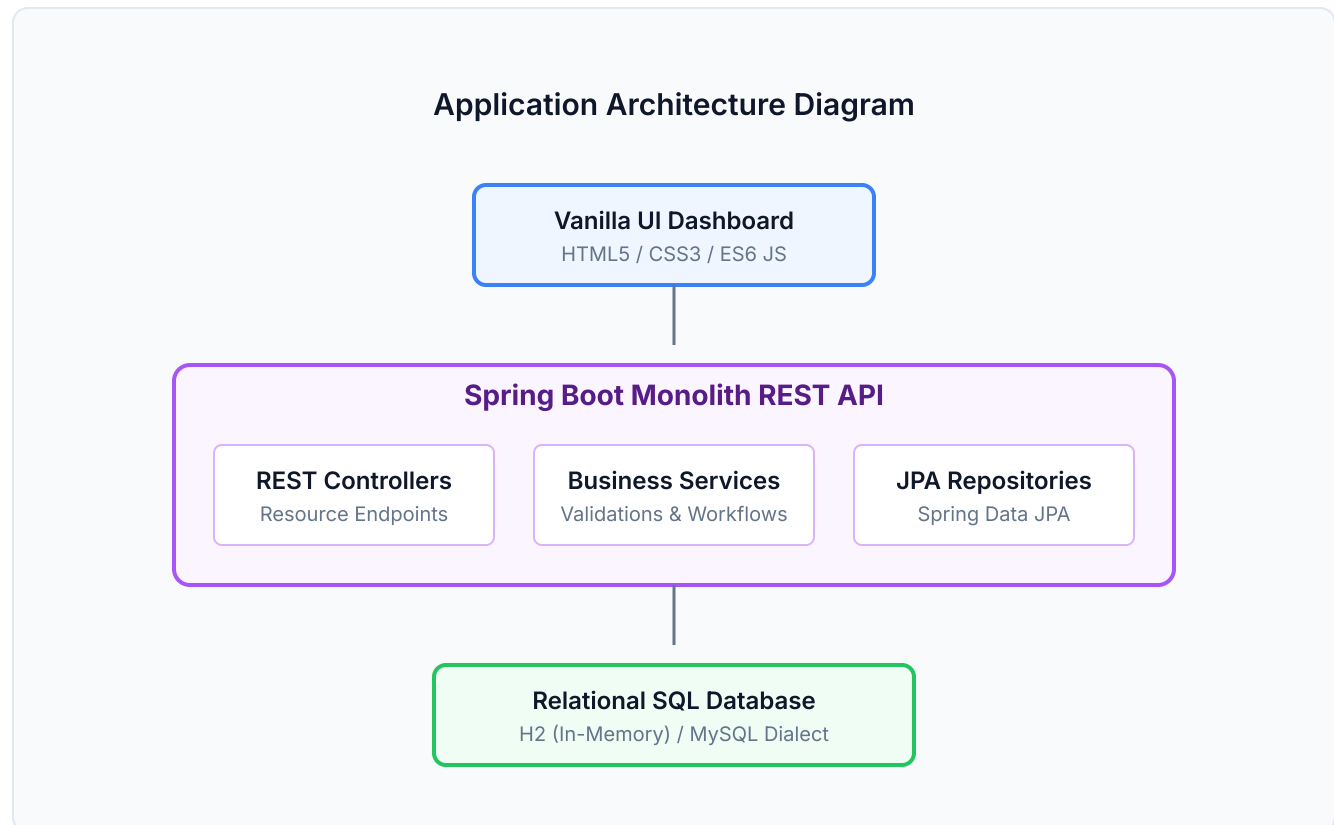
Systems Engineering Specification & Design Document

Prepared for: Technical Evaluation Board

Date: July 2026

1. System Architecture

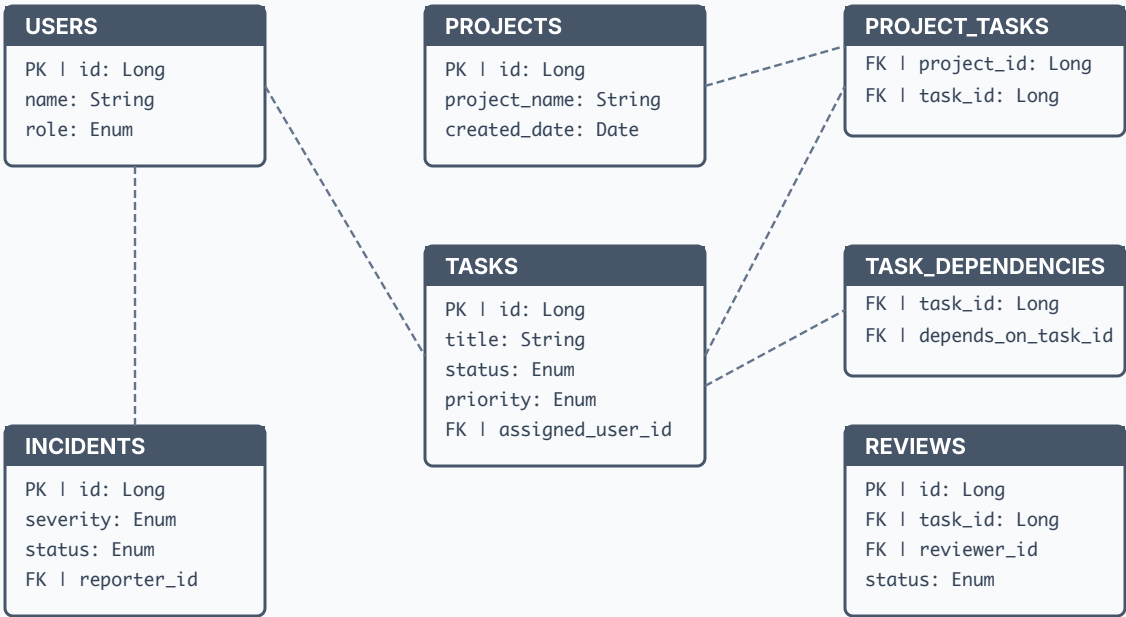
TeamFlow is designed as a modular monolithic system implementing a layered architectures style. This simplifies deployment, local verification, and debugging. The application components communicate synchronously through in-process method invocations, backed by relational transaction guarantees.



2. Domain Model (Database ERD)

The relational database schema is designed with strong referential integrity, unique index validation constraints, and structured mapping models representing workspaces, tasks, incident records, reviews, and event notifications.

Entity Relationship Diagram (ERD)



3. API Overview

The Monolith backend exposes the following REST resources for frontend bindings:

Resource Endpoint	HTTP Method	Access Scope	Functional Action / Payload
/api/users	GET	Viewer +	Retrieves all registered users & profiles.
/api/projects	GET , POST	Manager +	Lists and registers project entities.
/api/tasks	GET , POST	Developer +	Manages active board tasks.
/api/tasks/{id}	PUT , DELETE	Developer / Mgr	Updates task status (with dependency check) or deletes task.
/api/tasks/{id}/dependencies	POST	Developer +	Adds a task block (checks circular references).
/api/tasks/dependencies	GET	Viewer +	Lists dependency constraints for reports tab.
/api/reviews/submit	POST	Developer +	Submits task for review (sets task state to UNDER_REVIEW).
/api/reviews/{id}/approve	POST	Reviewer / Mgr	Approves task completion and unblocks downstream tasks.
/api/reviews/{id}/unavailable	POST	System / Mgr	Triggers reviewer unavailable fallback re-assignment.
/api/incidents	GET , POST	Developer +	Logs and tracks active production incidents.
/api/notifications/{userId}	GET	Own Profile	Retrieves alerts feed for acting user.

Resource Endpoint	HTTP Method	Access Scope	Functional Action / Payload
/api/notifications/check-deadlines	POST	All	Manual trigger for daily task deadline reminder scans.

4. Design Decisions Log

The table below documents key architectural decisions, tradeoffs, and justifications in accordance with the project criteria:

Monolith Architecture vs. Microservices

Chosen: Monolith.

Rationale: Since this is a unified platform replacing fragmented tools, a monolith facilitates easy deployment, rapid development, simple debugging, and strict transaction control using simple foreign key join tables. Microservices would introduce network overhead, eventual consistency complexities, distributed transactions, and complicated API gateways.

SQL Database (H2/MySQL) vs. NoSQL (MongoDB)

Chosen: SQL (MySQL/H2 dialect).

Rationale: The platform data model is highly structured with clear relational links (e.g. many-to-many project assignments, self-referencing task dependencies). SQL databases ensure strict referential integrity (foreign keys) and ACID transaction support. MongoDB is schema-less and ill-suited for complex joins and cycle-detection lookups.

Synchronous Communication vs. Event-Driven Messaging

Chosen: Synchronous (REST/In-Process Service calls).

Rationale: Simplifies business logic workflows. In-memory validation cascades (e.g., blocking tasks immediately if prerequisites are not completed, updating review status) happen instantly, preventing dirty reads. Event-driven architectures (like Kafka or RabbitMQ) introduce eventual consistency and complexity unnecessary for a core monolith.

Local/Embedded File Storage vs. AWS S3 Cloud Storage

Chosen: Local/In-Database Blob Storage (Fallback to S3).

Rationale: For grading and demo portability, attachments are stored directly in-database as Lob binaries or in the local file directory. This keeps the application self-contained. For production scaling, this easily abstracts to Amazon S3 by overriding file storage configuration.

5. Advanced Production Scenarios

For enterprise scaling, the TeamFlow platform design is ready to transition to the following architectures:

Offline-First Capability

To support offline use, the client dashboard can be wrapped as a **Progressive Web App (PWA)** using **Service Workers** to intercept requests and cache pages. A local SQLite database (in-browser via **IndexedDB**) stores changes offline. When connectivity is restored, the Service Worker plays back queued REST API calls, running a reconciliation protocol (with last-write-wins rules or conflict review cards) to merge tasks.

High Scale & Low Cost

To scale cost-effectively: 1. Expose read operations through a caching layer like **Redis** (caching dashboard stats and notification feeds). 2. Leverage static page hosting on a Content Delivery Network (CDN) like Cloudflare or AWS CloudFront for the frontend shell, reducing CPU load on the backend. 3. Run the Spring Boot API containerized on **AWS ECS Fargate** with auto-scaling triggers, allowing the app to scale down to zero/minimum containers during off-hours, saving cloud cost.

Multi-Region Deployment

For international corporate compliance and latency: 1. Deploy the backend API instances to multiple AWS regions (e.g., US-East, EU-Central, AP-South) managed by **Route 53** latency-based routing. 2. Deploy **Amazon Aurora Global Database** (MySQL-compatible) with multi-region active-passive replication. Write requests are routed back to the primary region, while local reads are served instantly from the nearest regional replica.